

术语表 · 忻的学习词典

一、环境与工程工具

终端 / Terminal

就是"用打字指挥电脑的黑窗口"。平时你用鼠标双击、点菜单，在终端里全靠敲命令。

- 打开方式：PyCharm 底部 `Terminal` 标签；或 `Win+R` → 输 `cmd` → 回车。
- 粘贴：Windows 终端里不能用 `Ctrl+V`，按鼠标右键粘贴。

venv / 虚拟环境 (virtual environment)

给每个项目单独准备的"食材冰箱"。项目 A 装的包只存在 A 的冰箱里，不会污染项目 B。

- 比喻：项目 A 的 `.venv` = 厨房 A 的冰箱；项目 B 的 `.venv` = 厨房 B 的冰箱。你在一个厨房找另一个厨房的食材当然找不到。
- 我们项目里装了 `requests` / `python-dotenv` / `openai`，WelcomeScreen 项目里没装，所以跨项目跑会报 `ModuleNotFoundError`。

PATH (环境变量)

一份"常用命令清单"，告诉电脑哪些文件夹里的程序可以直接用名字调用。

- 报错 `无法将"uv"识别为...` = 电脑的清单里找不到 `uv` 这个程序。
- 修法：把 `uv` 所在目录加进 `PATH` (已帮你加好)，或每次用 `python -m uv` 绕开。

uv

一个 Python 包管理 + 虚拟环境工具 (比老式的 `pip` + `virtualenv` 快很多)。

- `uv add requests` = 装 `requests` 包
- `uv run python xxx.py` = 在项目的虚拟环境里运行脚本
- 嫌麻烦可写成 `python -m uv run python xxx.py` (不受 `PATH` 影响，永远能用)

包 (package) / 模块 (module)

别人写好的、可以复用的代码集合。`import` 就是把别人的工具拿进来用。

- 比喻：SDK 是整套工具箱，模块是工具箱里的一个工具。
- 坑：`pip install` 的名字 ≠ `import` 的名字 (见下条)。

pip install 名 vs import 名

PyPI 上的项目名 (安装用) 和 Python 里的模块名 (导入用) 常常不一样。

<code>pip install</code> 时	<code>import</code> 时
<code>python-dotenv</code>	<code>import dotenv</code>
<code>PyYAML</code>	<code>import yaml</code>
<code>Pillow</code>	<code>from PIL import Image</code>
<code>scikit-learn</code>	<code>import sklearn</code>
<code>BeautifulSoup4</code>	<code>from bs4 import BeautifulSoup</code>

硬编码 (hardcode)

把本该可变的"密码 / 配置"直接写死在代码里。

- 反面教材：`api_key = "sk-5cd26..."` (密钥暴露在代码里，传到 GitHub 就泄露)
- 正确做法：放 `.env`，代码里 `os.getenv("DEEPSEEK_API_KEY")` 读。

.env

专门放密码的"记事本"文件。名字来自 environment（环境）。

- 内容: `DEEPSEEK_API_KEY=sk-xxxx`
- 代码里: `os.getenv("DEEPSEEK_API_KEY")` 读取
- `.env` 被 `.gitignore` 排除, 永远不会被提交到 GitHub, 所以安全
- `.env.example` 是模板 (里面是空的), 可以传, 别人复制它改名 `.env` 填自己的

SDK (Software Development Kit)

别人替你写好的一套现成工具 ("开发工具包")。

- 比喻: 要开电视, `requests` 是自己接电线调信号; SDK 是拿遥控器按一下。
- `openai` 这个 SDK 故意和 DeepSeek 兼容, 所以 `from openai import OpenAI + base_url="https://api.deepseek.com"` 能直接调 DeepSeek。
- 生产环境用 SDK (自带重试/限流/超时); 学 `requests` 是为了懂原理。

二、Python 数据类型

字典 dict

键值对集合。 `{"name": "玉环", "code": "331083"}`

- 取值: `d["name"]` 或 `d.get("name")`
- 键必须唯一; 值可以是任意类型 (包括另一个字典 → 嵌套)

列表 list

有序的元素集合。 `[1, 2, 3]`

- 取值: `lst[0]` (从 0 开始)
- 追加: `lst.append(x)`
- 可变: 造出来还能改

元组 tuple

和列表像, 但不可变。 `(1, 2, 3)`

- 用圆括号 `()`; 列表用方括号 `[]`
- 没有 `.append()`, 造出来就锁死
- 你 Tier 1 写 `qw = (...)` 就是错把元组当列表, 导致 `.append()` 报错

f-string

字符串里嵌套变量的写法。字符串前加 `f`, `{}` 里放表达式。

```
name = "忻"
print(f"你好 {name}")      # 你好 忻
print(f"明年 {28 + 1} 岁")  # 明年 29 岁
```

- 忘了写 `f` → `{}` 原样显示, 不报错 (最难发现的 bug)
- 格式说明符: `{x:.2f}` (2位小数)、`{x:.1%}` (百分比)、`{x:,}` (千位分隔)、`{x:>10}` (右对齐宽10)
- 详见 [week01/notes/fstring_cheatsheet.md](#)

三、取数与错误处理

嵌套取值

一层套一层地从字典里掏数据。

```
data["choices"][0]["message"]["content"]
# 字典      列表      字典      键
```

- 看不懂就拆开一层一层 `print`，拆三次就会了。
- 风险：任意一层 `key` 不存在 → `KeyError` 崩溃。用 `.get()` 化解。

`.get()`

字典的“安全取值”。取不到也不崩。

```
d.get("age")          # 没有 → None（不报错）
d.get("age", 0)       # 没有 → 返回 0
d.get("a", {}).get("b", {}).get("c")  # 链式，中间用 {} 兜底保证链不断
```

- 拿到 `None` 后不能直接拼字符串（`"x" + None` 崩），要先 `if name:` 判断。
- 详见 [week01/notes/get_and_tryexcept_cheatsheet.md](#)

`try / except`

出错也不让程序挂。`try` 放可能崩的代码，`except` 放崩了怎么办。

```
try:
    resp = json.loads(raw)
except json.JSONDecodeError:
    print("解析失败")
else:
    print("解析成功")          # 没崩才走
```

- 该写哪种错误名？看报错最后一行最下面的名字：`KeyError` / `IndexError` / `TypeError` / `json.JSONDecodeError` ...
- 初学先用 `except Exception:` 兜底，跑通再换具体名。
- 详见 [week01/notes/get_and_tryexcept_cheatsheet.md](#)

常见报错名（背下来，看到就不慌）

报错	含义	常见原因
<code>KeyError</code>	字典里没有这个键	<code>d["不存在的key"]</code>
<code>IndexError</code>	列表下标越界	<code>lst[10]</code> 但只有 3 个
<code>TypeError</code>	类型不对	<code>"x" + None</code> 、 <code>1 + "2"</code>
<code>AttributeError</code>	对象没有这个方法	<code>None.get(...)</code> 、 <code>(1, 2).append(...)</code>
<code>JSONDecodeError</code>	不是合法 JSON	<code>json.loads("垃圾文本")</code>
<code>ModuleNotFoundError</code>	没装这个包	跨 <code>venv</code> 跑 / 没 <code>uv add</code>
<code>IndentationError</code>	缩进错	多余空格 / 混用 <code>tab</code> 和空格

四、接口与数据

API (Application Programming Interface)

两个程序之间“对话”的约定。你写代码调 DeepSeek 的 API，就是按它的格式发请求、收回答。

- 比喻：餐厅里服务员是 API——你（程序 A）点单，服务员把单传给厨房（程序 B），再把菜端回来。

JSON (JavaScript Object Notation)

一种“用文本表示数据”的通用格式。接口来回传的都是它。

```
{"name": "玉环", "code": "331083"}
```

- 对应 Python：长得像字典，但它是文本（字符串），要先 `json.loads` 才能当字典用。

`json.loads` / `json.dumps`

`json` 模块的两个核心函数（一对）。

- `json.loads(s)` = load string：把 JSON 文本 → Python 字典（读进来）

- `json.dumps(obj)` = dump to string: 把 Python 字典 → JSON 文本（写出去）
- 你 Day 1 的 `resp.json()` 底层就是 `json.loads`; `requests.post(json=xxx)` 底层就是 `json.dumps`。

raw

"原始的、未加工的"数据，通常是个变量名。

```
raw = '{"data": {"city": {"name": "玉环"}}}' # 还没解析的 JSON 文本
```

- 下一步 `json.loads(raw)` 把它"加工"成字典。加工前的叫 raw。

resp

"response"（响应）的缩写，通常装"接口返回的数据"。

```
resp = json.loads(raw) # resp 是解析后的字典
```

- 就是个普通变量名，叫 `data` / `result` 也行。
- 你 Day 1 的 `data = resp.json()` 里 `data` 和它扮演同一角色。

requests

一个发 HTTP 请求的 Python 库（自己接电线那套）。

```
import requests
r = requests.post(url, json={"k": "v"}) # 发 POST
r.json() # 把返回文本 → 字典
```

- 和 SDK 对比：SDK 是遥控器，requests 是自己接线。学它是为了懂原理。

序列化 / 反序列化

数据在"内存对象"和"可传输文本"之间来回变。

- 序列化（对象 → 文本）：`json.dumps`、DeepSeek 请求的 `json=`
- 反序列化（文本 → 对象）：`json.loads`、`resp.json()`
- Day 1 的"六步数据变形"本质就是这两步在跨机器往返。

五、一句总提醒

看到不认识的词，先问自己三件事： 1. 它是变量名（随便起的）还是关键字（Python 固定的）？ 2. 它是模块名（要 `import`）还是函数名（要 `()` 调用）？ 3. 它在做"取值"、"转换"还是"控制流程"？ 分不清就发我那个词，我拆。

六、LLM 与 Agent 核心词汇（预习，按学习阶段排）

这些是你后面几周一定会撞上的。现在不要求懂，先有个印象，撞到时回来搜。

A. 大模型基础（W6 前后）

token（词元） 模型读写的最小计价单位，不是"字"也不是"词"。中文约 1-2 字 = 1 token。你 DeepSeek 返回的 `usage` 里 `prompt_tokens` / `completion_tokens` 就是它。计费、限速、上下文长度都按 token 算。

prompt（提示词） 你发给模型的指令文本。包括 `system` / `user` / `assistant` 三种角色的消息。

system prompt（系统提示）

最前面的一条指令，设定模型的"身份和规矩"。比如"你是政务助手，只回答公开政策"。你 Day 2 的聊天机器人没写 `system`，所以模型不知道自己是谁。

temperature（温度） 控制输出随机性。0 = 每次都一样（稳定），1 = 更发散（有创意）。写代码/抽字段用 0，写文案用高一点。

top_p 和 temperature 类似的"随机性"旋钮，按概率截断。一般只调其中一个，不两个都动。

few-shot / zero-shot（少样本 / 零样本） zero-shot = 不给例子直接让模型做；few-shot = 在 prompt 里先给几个"示例问答"再让它做。抽字段、分类任务用 few-shot 准确率暴涨。

上下文窗口（context window） 模型一次能“看到”的最大 token 数（输入+输出合计）。超出会被截断遗忘。DeepSeek 一般是 32k/64k 级别。

幻觉（hallucination） 模型一本正经地编假内容。

RAG（见下）主要就是用来压幻觉的——给它真文档，让它照着说。

B. 提示工程（W6）

提示工程（prompt engineering） 通过设计 prompt 让模型稳定产出想要结果的技术。2026 年 JD 里“上下文工程”比它更值钱（见 F 节）。

C. RAG 检索增强生成（W10-13）

嵌入 / embedding 把一段文字变成一串数字（向量），语义相近的文字向量也相近。这是“让机器懂语义”的基础。

向量 / vector 嵌入产出的那串数字。存在向量库里，用来算相似度。

切片 / chunk 把长文档切成小块再喂给模型——模型一次看不下整本手册，先切。切多大、怎么切是 RAG 效果的关键。

文档解析 / parsing 把 PDF/Word/网页变成纯文本的过程。表格、图片常在这里丢信息，是 RAG 的常见坑。

向量库（FAISS / Milvus / Chroma） 专门存向量、做“找最相似”的数据库。你计划里 RAG 阶段要装一个。

召回 / recall 用户提问后，从向量库找出“最相关几段”的过程。召回准不准决定答案下限。

重排 / rerank 召回后，用更精的模型把候选再排一遍序，把最相关的顶到前面。你计划里“纯向量 vs +Rerank”的对照实验就在这。

余弦相似度（cosine similarity） 衡量两段向量（语义）有多像的指标，值域 $-1 \sim 1$ ，越接近 1 越相关。向量库检索的底层算法。

RAG（检索增强生成，Retrieval-Augmented Generation） 先检索相关资料、再让模型基于资料回答的架构。企业 70% 的 AI 落地本质都是 RAG。目的：压幻觉、用私有知识。

D. Function Calling / 工具调用（W7-9）

Function Calling（函数调用 / 工具调用） 让模型决定“该调哪个工具、传什么参数”的能力。

比如模型判断“用户问天气→调 `get_weather(city)`”。这是 Agent 能“动手”的根本。

tool use（工具使用） Function Calling 的同义说法。工具 = 你写的一个函数（查数据库、调 API、算账……）。

JSON Schema

描述“一个函数长啥样”的规范格式——名字、参数是什么类型、是否必填。你要把工具告诉模型，就得写它的 JSON Schema。

参数 schema JSON Schema 里描述“这个函数要哪些参数”的部分。模型输出会严格按这个结构来（你也才能 `.get()` 取值）。

重试 / retry 调用失败（超时、限流、模型抽风）后自动再试一次。生产必备。

退避 / backoff 重试时“等一会再试”，且越试等越久（如 $1s \rightarrow 2s \rightarrow 4s$ ），避免把对方服务器打爆。常见叫“指数退避”。

E. Agent 架构与上下文工程（W14-17）

ReAct 一种 Agent 经典套路：Reason（思考）→ Act（调工具）→ Observe（看结果）→ 再 Reason……循环。你 W7-9 要手写这个循环。

Agent loop（智能体循环） 上述“思考-行动-观察”反复跑，直到任务完成的整个过程。你 Day 1 说“一知半解”的那一层，最后会落脚在这。

LangGraph 一个用“图”来编排 Agent 流程的框架（节点=步骤，边=走向）。你 W14 开始用，之前手写的循环它帮你封装好了。

状态 / state Agent 在循环里“记得的东西”（对话历史、中间结果、当前步骤）。图里数据在节点间靠 state 传递。

节点 / node、图 / graph 图编排的两个概念：node = 一个处理步骤（如“调模型”“查库”），graph = 这些步骤怎么连起来、谁先谁后。

checkpoint（检查点）把 Agent 运行到一半的状态存下来，出错能从中间恢复而不是重来。你计划里“可恢复 Agent”就靠它。

上下文工程（context engineering）2026 年 JD 高频词：比“提示词工程”更值钱。

指系统性地管理“喂给模型的全部信息”（历史、工具描述、检索结果、约束），而不只是写一句 prompt。

F. 生产工程（W18-21）

评测集 / eval 一套标准题目+标准答案，用来量化“你的 Agent 好不好”。“召回从 61%→89%”这种数字就来自评测集。作品集的核心证据。

基准 / benchmark 业界公认的标准评测（如 MMLU）。个人项目一般用自己做的 eval 集，不必追 benchmark。

观测 / observability、链路追踪 / tracing 把每次调用的“模型输入输出、耗时、花了多少 token、卡在哪一步”记下来看。排错和优化的眼睛。

限流 / rate limit 接口方限制你“单位时间能调多少次”（你 Day 2 就撞过 429）。靠重试+退避应对。

提示词注入 / prompt injection、越狱 / jailbreak

攻击者骗模型无视规矩干坏事（如在文档里藏“忽略上面所有指令”）。做对外 Agent 必须防。

对齐 / alignment

让模型行为符合人类预期与安全规范的研究方向。你做产品时会遇到“模型该不该拒绝某类问题”的取舍。

流式响应 / streaming 模型边生成边往回传，你看到打字机效果。聊天产品标配，比“等全生成完再给”体验好。

七、通用软件工程词汇（预习）

函数 / function 一段可重复调用的代码。def add(a, b): return a + b 就是函数。你现在写的 for 循环里那段逻辑，将来会包成函数复用。类 / class、面向对象 / OOP

把“数据”和“操作数据的方法”打包成一个“对象”的写法。你做 Agent 时 class ChatBot:

会比一堆零散函数好管。初学先会写函数，类晚点再碰。装饰器 / decorator

在不改函数内部的情况下，给它“加功能”的语法，形如 @retry。你将来会写 @app.post("/chat")

注册接口——那就是装饰器。类型注解 / type hints 给变量/函数参数标类型，如 def f(x: int) -> str:。不强制但能帮 IDE 报错、帮人读懂。FastAPI 强烈依赖它。异步 / async、await “等

IO（网络/磁盘）时不干等，先去干别的”的写法。FastAPI 接口常用 async def。你 W1-5

会学，先知道它解决“高并发不卡”就行。并发 / 并行 并发 = 多件事交替推进（像单核切时间片）；并行 =

真同时跑（多核）。调多个工具时常用到。阻塞 / 非阻塞 阻塞 = 等结果期间啥也干不了；非阻塞 =

等的时候能去干别的。async 就是非阻塞思路。日志 / logging 用 logging 模块把程序运行过程写进文件，比 print 专业（能带时间、级别、存盘）。排线上问题靠它。依赖 / dependency

你的项目“依赖”的第三方包（requests、openai...）。uv add 就是加依赖。缓存 / cache

把算过/取过的结果暂存，下次直接用，省时间省钱。DeepSeek 返回的 prompt_cache_hit_tokens

就是命中缓存——你 Day 1 见过。超时 / timeout

“等对方回应最多等几秒，超了就当失败”。不设这个，对方卡死你会跟着卡死。幂等 / idempotent

同一个操作重复做多次，结果一样、不产生副作用。比如“扣款”必须幂等，否则重试一次扣两次。HTTP 方法 GET / POST GET = 去拿数据（查）；POST = 提交数据（改/建）。你 Day 1 调 DeepSeek 用的是 POST。状态码 200 / 400 / 401 / 429 / 500 接口回应的“结果代号”：200=成功，400=请求错，401=没权限（Key错），429=太快被限流，500=对方服务器炸。你看报错先看这个数字。Git 基础 版本控制工具。

commit=存快照，branch=开分支，push=推到远程，pull=拉下来，PR=请求合并。你 Day 1 已用 commit，后面要学 push 到 GitHub。部署 / deploy、Docker、云 把你写的程序放到“别人能访问的服务器”上跑。

Docker=把程序连环境打包成容器；云=租的服务器（阿里云/腾讯云）。作品集最后要部署出来才叫“上线”。

八、怎么用这份表

1. 撞到不认识的词 → Ctrl+F 搜。搜不到就发我，我补进去。2. 第六、七节是预习，不用现在背。

学到那一周再回来细看对应的段。3. 真正的懂 = 用过一次。

每个词你都会在实际代码里撞到，撞到时才算真记住。